Fast Gaussian process regression via low-rank approximations

Junhyung Chang Shrikant Chand

New York University

December, 2021

Overview

Motivation

- In Rasmussen & Williams, 2006, the posterior distribution of the outputs $\mathbf{y}^*$ conditioned on $\mathbf{x}, \mathbf{y}, \mathbf{x}^*$ is shown to be

$$\mathbf{y}^* | \mathbf{x}, \mathbf{y}, \mathbf{x}^* \sim \mathcal{N}(\overline{\mathbf{y}}^*, \mathrm{cov}(\mathbf{y}^*)) \tag{1.1}$$

  where

$$\overline{\mathbf{y}}^* = K(\mathbf{x}^*, \mathbf{x})[K(\mathbf{x}, \mathbf{x}) + \sigma^2 I]^{-1}\mathbf{y}, \tag{1.2}$$

$$\mathrm{cov}(\mathbf{f}^*) = K(\mathbf{x}^*, \mathbf{x}^*) - K(\mathbf{x}^*, \mathbf{x})[K(\mathbf{x}, \mathbf{x}) + \sigma^2 I]^{-1}K(\mathbf{x}, \mathbf{x}^*). \tag{1.3}$$

- How to compute

$$[K(\mathbf{x}, \mathbf{x}) + \sigma^2 I]^{-1}$$

  when system size is large ($\sim 10^6$)?

Overview

Three approaches

1. Global randomized low-rank factorization of K.
2. Truncated eigenfunction expansion of integral operator $\mathcal{K}$.
3. Matrix compression using hierarchical rank structure of $K(\mathbf{x}, \mathbf{x}) + \sigma^2 I$.

Approach 1

Global randomized low-rank factorization. $O(n^2\ell)$.
The main reference is Martinsson, 2019.

1. Stage A
   - Find approximate range of matrix K
   - by drawing random matrix $\Omega \in \mathbb{R}^{n\times\ell}$, where ℓ is the target rank + oversampling parameter. Ω has full rank almost surely.
   - Hence, $Q := \mathrm{orth}(K\Omega)$ is an approximate orthonormal basis for the range of K.
   - K is approximated by the projection $K \approx QQ^*K$.
   - Stage A contains most of the computational work, but can be easily parallelized.
2. Stage B
   - E.g., let $U = Q$ and $V = K^*Q \implies K \approx UV^*$.
   - A single-pass algorithm can be employed to find low-rank factors without having to evaluate K again.

Approach 1

- Stage B (cont'd)
  - For Hermitian matrix K, $K \approx QQ^*K = KQQ^* = QQ^*KQQ^*$.
  - Take eigenvalue decomposition of $C = Q^*KQ = \hat{U}D\hat{U}^*$,then the ev-decomposition of K is $(Q\hat{U})D(Q\hat{U})^* := UDU^*$.
- Then, inverting $I + K$ becomes

$$(I + UDU^*)^{-1} = U(I_\ell + D)^{-1}U^*.$$

- Note: Stage A can be further accelerated by choosing structured Ω, e.g. via the fast Johnson-Lindenstrauss transform.
- Approach 1 scales quadratically with n, but is easily parallelizable.

Approach 2

Eigenfunction expansion of integral operator $\mathcal{K}$. $O(nm^2)$.

- Let a GP f be approximated by a truncated series

$$f \approx \sum_{i=1}^{m} \alpha_i g_i,$$

where $\alpha_i \sim \mathcal{N}(0,1)$. Then, it can be shown that

$$k(x,x') = \mathrm{cov}(f(x), f(x')) \approx \sum_{i=1}^{m} g_i(x) g_i(x') := \hat{k}(x,x').$$

- By choosing g_i as the Karhunen-Loéve (KL) expansion basis functions, the kernel $\hat{k}$ of the order-m expansion is the best order-m approximation of the true kernel k in the L^2 norm.

Approach 2

- Moreover, as $m \to \infty$, the convergence is uniform and absolute due to Mercer's theorem.
- Brief explanation: define the integral operator

$$\mathcal{K}f(x) = \int_{-1}^{1} k(x, x')f(x')dx'.$$

- The KL-expansion basis functions are

$$\varphi_i(x) := \sqrt{\lambda_i}u_i(x),$$

  where λ_i's are eigenvalues of $\mathcal{K}$ in decreasing order, and $u_i(x)$'s the corresponding normalized eigenfunctions.
- λ_i, $u_i(x)$ are found by discretizing the integral operator $\mathcal{K}$ using standard techniques. See e.g., Greengard & O'Neil, 2021.

Approach 2

- Since each entry of $K(\mathbf{x}, \mathbf{x}')$ is approximated by $\sum_{i=1}^{m} \varphi_i(x)\varphi_i(x')$,

$$K(\mathbf{x}, \mathbf{x}') \approx XX^*,$$

  where $X = (\varphi_i(x_j))_{ij}$.
- Note: Approach 2 is $O(nm^2)$, hence scales **linearly** with n.
- However, additional errors are introduced due to discretization.
- Also, difficult to scale to higher dimensions ($d > 2$) due to the curse of dimensionality.

Approach 3

Matrix compression using hierarchical rank structure. $O(n \log^2 n)$.

- Key observation: for many covariance functions, the rank of $(I + K)$ is "concentrated" "near" the diagonal.
- Compress the highly low-rank off-diagonal blocks.
- Divide $A := I + K$ into four blocks:

$$A = \left[\begin{array}{c|c} A_{11} & A_{12} \\ \hline A_{21} & A_{22} \end{array}\right].$$

- Compress low-rank blocks $A_{12} = UV^*, A_{21} = VU^*$. Then the inverse can be computed efficiently via Woodbury matrix identity, since

$$A = \hat{A} + \hat{U}\hat{V}^* = \begin{bmatrix} A_{11} & 0 \\ 0 & A_{22} \end{bmatrix} + \begin{bmatrix} 0 & U \\ V & 0 \end{bmatrix} \begin{bmatrix} V^* & 0 \\ 0 & U^* \end{bmatrix}.$$

Approach 3

- If the block diagonals A_{ii}'s also have off-diagonal low-rank structure, A can be compressed even further.
- an efficient way to invert the compressed matrix is to construct HODLR factorization ($O(n \log^2 n)$).

$$A = A_\kappa A_k A_{k-1} \dots A_1.$$

- For example, a one-level factorization is

$$A = A_\kappa A_1 = \begin{bmatrix} A_{11} & 0 \\ 0 & A_{22} \end{bmatrix} \begin{bmatrix} I & A_{11}^{-1} A_{12} \\ A_{22}^{-1} A_{21} & I \end{bmatrix}.$$

- Repeat the procedure on the block diagonals of A_κ to construct further levels.
- Compute $A^{-1} = A_1^{-1} \dots A_\kappa^{-1}$.

Approach 3

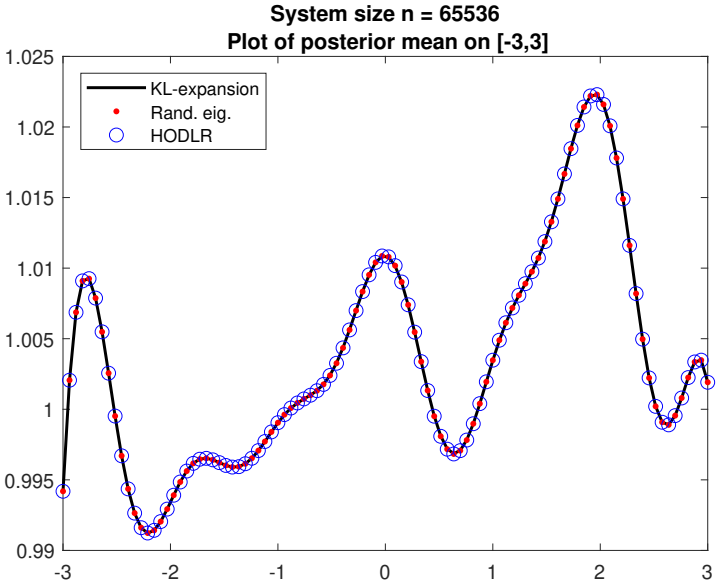
- Only the low-rank factorizations $U_i^{(j)}, V_i^{(j)} \in \mathbb{C}^{n \times r}$ of off-diagonal blocks should be stored.
- Starting from A_κ, once the inverse of a block diagonal $A_{k(ii)}^{-1}$ is computed via the Woodbury identity, it should be applied immediately to all of the remaining $U_i^{(j)}, V_i^{(j)}$'s $(j < k)$ *at the right locations*, and to $\mathbf{y}$ in the case the ultimate goal is to compute the posterior mean. This way, the diagonal blocks need not be stored.
- Typically, computing the low-rank approximations of off-diagonal blocks takes up the vast majority of the run-time. Forming and inverting the actual HODLR factorization is very fast.
- Choice of low-rank approximation for off-diagonal blocks is important; this strongly affects the run-time.

Numerical experiments

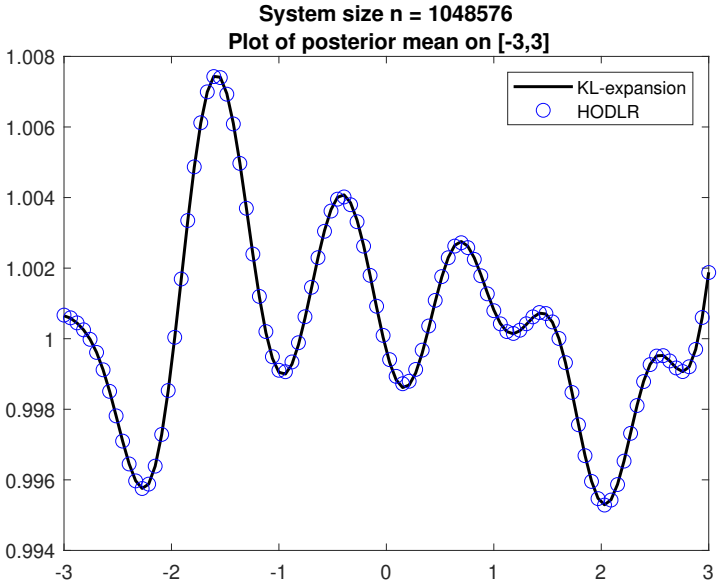
Setup

- Training inputs $\mathbf{x}$ were chosen randomly on $[-3, 3]$, and training outputs $\mathbf{y}$ were chosen randomly on $[0, 2]$.
- Test inputs $\mathbf{x}^*$ were chosen as 100 equispaced points on $[-3, 3]$.
- The Gaussian covariance function $k(x, x') = e^{-|x-x'|^2}$ is used for the following plots and tables.

Numerical experiments



Numerical experiments



Numerical experiments

n	Stage A	Stage B	Invert
2^8	0.0117	0.0023	0.0022
2^{10}	0.0727	0.0005	0.0018
2^{12}	0.5718	0.0013	0.0004
2^{14}	7.2198	0.0044	0.0012
2^{16}	106.8086	0.0142	0.0072
$2^{18} \approx 2 \times 10^5$	1974.8465	0.0436	0.0286

Figure: Run-time in seconds for method 1 (randomized SVD)

Numerical experiments

n	KL-expand	Factor & invert
2^8	1.1837	0.0810
2^{10}	1.1125	0.3377
2^{12}	0.9705	1.4091
2^{14}	1.2947	6.1126
2^{16}	0.9394	19.7492
2^{18}	1.0667	79.3289
$2^{20} \approx 10^6$	0.8980	332.6400

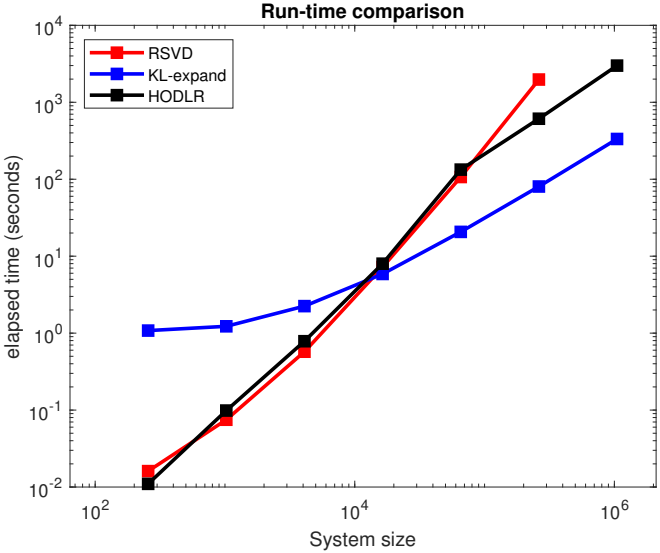
Figure: Run-time in seconds for method 2 (KL-expansion)

Numerical experiments

n	Compress	Invert
2^8	0.0236	0.0391
2^{10}	0.0697	0.0378
2^{12}	1.0019	0.2265
2^{14}	8.0835	1.3945
2^{16}	127.6205	5.7087
2^{18}	581.8549	29.4437
$2^{20} \approx 10^6$	2844.7266	150.3182

Figure: Run-time in seconds for method 3 (HODLR factorization)

Numerical experiments



Conclusion

Future directions

- Incorporate different covariance functions.
- In-depth error estimation.
- Projection onto structured range-finders other than Gaussian random matrix.
- Performing hyperparameter (maximum likelihood) estimation using these results.
- Performing GP classification using these results.